

Slide 1: Title Slide

Speaker Notes:

“Good morning, everyone. My name is Abdulla Husain Salem Hadna Al Messabi, and I’m here today to present my project on the design, training, and evaluation of a Convolutional Neural Network (CNN) that I built to classify images from the CIFAR-10 dataset. This project was completed as part of the Deep Learning track of my course, and the focus is on using deep learning techniques, specifically CNNs, to tackle image classification.

Before we dive into the technical details, I’ll walk you through an overview of the dataset I worked with, the architecture of the CNN model, and the results I achieved. I will also discuss some of the challenges I faced and the lessons I learned along the way. So, let’s get started.”

Slide 2: Project Overview

“The primary objective of this project was to build and evaluate a CNN model to classify images from the **CIFAR-10 dataset**, which is one of the most widely used benchmarks for image classification tasks. CIFAR-10 is a dataset that contains 60,000 images in 10 distinct object categories, such as airplanes, cars, and cats, among others. The goal was to design a CNN that could classify these images as accurately as possible.

For this project, I focused on **Deep Learning (Track 2)**, which specifically involves Convolutional Neural Networks. CNNs have proven to be highly effective in image recognition tasks because they can automatically learn hierarchical patterns directly from raw pixel data, eliminating the need for manual feature extraction. Unlike classical machine learning models that depend on human-engineered features (like edges, textures, or shapes), CNNs learn complex representations through layers of convolutions, pooling, and activation functions.

I’ll now guide you through the dataset, the model architecture, the training process, and finally, the results that I achieved in this project.”

Slide 3: Dataset Overview

“The **CIFAR-10 dataset** is a standard benchmark for evaluating machine learning models, especially in the realm of image classification. The dataset consists of **60,000 images**, each with a size of **32x32 pixels** and **3 color channels**—RGB (Red, Green, Blue). These images are divided into **10 distinct classes**, with each class having **6,000 images**. The classes are as follows: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

The dataset is well-balanced, meaning each class contains an equal number of images. Out of the 60,000 images, **50,000 images** are used for training the model, and the remaining **10,000 images** are reserved for testing. To avoid overfitting, I further split the training data into **45,000 images for training** and **5,000 images for validation**. This separation allows the model to be evaluated on unseen data during training, giving insights into how well it generalizes to new data.

Here's an overview of the classes in CIFAR-10. As you can see, there is a diverse range of objects, from animals like cats and dogs to vehicles like trucks and airplanes. The variety in the dataset makes it a challenging task for the model, as the images in each class are quite varied and can often resemble objects from other classes, such as the similarity between dogs and cats or between cars and trucks."

Slide 4: Data Preparation and Augmentation

"The preparation of the data is one of the most crucial steps in any machine learning project, and it's especially important when working with image data. First, I started by **normalizing the pixel values**. In image data, the pixel values typically range from 0 to 255, which is not ideal for deep learning models. To help the model train more efficiently, I converted each pixel value to a **floating-point number** and then **normalized** it using the mean and standard deviation of the training set. This ensures that the input data is scaled properly, improving the training process and speeding up convergence.

Additionally, I split the data into **training**, **validation**, and **test** sets. As mentioned earlier, 45,000 images were used for training, 5,000 for validation, and 10,000 images were used for testing. The validation set allowed me to monitor the model's performance during training and adjust hyperparameters accordingly.

One of the key techniques I employed to improve the model's generalization and avoid overfitting was **data augmentation**. This involves artificially creating new training data by applying random transformations to the original images. I used several types of augmentation, including **rotation**, **shifting**, **flipping**, **zooming**, **brightness adjustments**, and **channel shifts**. By introducing these variations, I ensured that the model would learn to recognize objects in different orientations and lighting conditions. This enhanced the model's robustness and helped it generalize better to real-world scenarios, where images might not always appear in exactly the same way as in the training set.

The result of these augmentations was a more diverse training set, which enabled the CNN to learn more robust features, thus improving the accuracy and performance of the model."

Slide 5: Validation Strategy

"A key challenge in training deep learning models is ensuring that they don't overfit to the training data. Overfitting occurs when a model learns to memorize the training data instead of learning general patterns that can be applied to new, unseen data. To combat this, I used a **validation strategy** that helped monitor overfitting and allowed me to fine-tune the model during training.

During training, I used a separate **validation set** that was not part of the training data. This allowed me to evaluate the model's performance at each epoch and see how well it generalized to

data it had not seen before. This is critical for adjusting hyperparameters, such as the learning rate, to ensure that the model doesn't overfit.

To further prevent overfitting, I implemented **early stopping**. This technique halts the training process if the validation loss stops improving for a set number of epochs—in this case, **40 epochs**. Early stopping helps save computational resources and avoids the model from continuing to train when no meaningful improvements are being made.

Additionally, I employed **learning rate scheduling**, specifically the **ReduceLROnPlateau** callback. This technique reduces the learning rate when the validation performance plateaus. By reducing the learning rate, the model is able to fine-tune its weights more delicately and reach a better solution.”

Slide 6: CNN Architecture

“Now, let's dive into the architecture of the CNN. The core of the model consists of four convolutional blocks, each of which is designed to learn increasingly complex features in the image. After these blocks, the output is flattened and passed through a dense layer that classifies the image into one of the ten classes.

Here's a breakdown of the architecture:

- **Block 1:** In this block, I used **32 filters** for the convolutional layer, followed by **batch normalization**, **max pooling**, and a **dropout rate of 0.2**. The batch normalization layer helps stabilize training, and the max pooling layer reduces the spatial dimensions of the image, making the model more computationally efficient.
- **Block 2:** The second block had **64 filters**, batch normalization, max pooling, and a **dropout rate of 0.3**. As the model progresses deeper, the number of filters increases to capture more complex features in the image.
- **Block 3:** This block used **128 filters**, batch normalization, max pooling, and a **dropout rate of 0.4**.
- **Block 4:** The final block had **256 filters**, batch normalization, max pooling, and a **dropout rate of 0.5**. The increasing dropout rates help control overfitting as the model gets deeper and more complex.

Finally, the output of the four convolutional blocks is **flattened** and passed through a **dense layer** with **10 units** corresponding to the 10 classes in the CIFAR-10 dataset. The activation function used here is **Softmax**, which gives the probability distribution over the classes.

In addition, I used **L2 regularization** with a weight decay of **1e-4** to prevent large weights that could lead to overfitting. The **Adam optimizer** was chosen for its ability to handle sparse gradients and adjust the learning rate dynamically during training.”

Slide 7: Training Strategy

“Training the CNN involved running the model over up to **300 epochs**. However, thanks to **early stopping**, the training process often stopped earlier, saving both time and computational resources.

I used a **batch size of 64**, which is a good balance between memory usage and training speed. The **Adam optimizer** was used with a learning rate of **5e-4**, which helped the model converge efficiently.

Three key callbacks were used during training to ensure the process was both efficient and stable:

1. **ReduceLROnPlateau**: This callback automatically reduced the learning rate when the validation performance plateaued, allowing the model to escape local minima and fine-tune the weights.
2. **EarlyStopping**: This callback stopped the training if the validation loss did not improve for 40 epochs, which helped prevent overfitting.
3. **ModelCheckpoint**: This callback saved the best model weights based on validation performance, ensuring that I had the best-performing model at the end of training.

With these strategies in place, the training was stable and the model was able to converge effectively.”

Slide 8: Performance Evaluation

“Now, let’s look at the performance of the model. After training, the CNN achieved an impressive **test accuracy of 91.03%** and a **test loss of 0.4118**, which indicates that the model performed well in classifying the test images.

The training accuracy was around **92.5%**, and the validation accuracy was **91.1%**. This close match between training and validation accuracy suggests that the model did not overfit and was able to generalize well to unseen data. The **confusion matrix** shown here visualizes how the model performed across all ten classes.

As you can see from the matrix, most of the classes were classified correctly, though there were some misclassifications, particularly between visually similar classes, such as cats and dogs. These misclassifications are expected, as the model is still learning to distinguish between objects that share similar features.

To further evaluate the model, I ran an **inference test** on a sample image of a truck, and the model successfully classified it correctly as a truck. This demonstrates the model’s ability to generalize well on new, real-world images.”

Slide 9: Inference Example

“Here, you can see an example of the model making an inference on a **single truck image**. The model predicted the class as **truck**, which is the correct classification. This example shows the strength of the model in terms of generalization. Even though this image wasn’t part of the training data, the model was able to correctly classify it. This is a good sign that the model is capable of recognizing truck images from a wide range of inputs.”

Slide 10: Comparative Discussion

“Now, let's compare the CNN model I used with other machine learning techniques.

1. **Classical ML:** Approaches like **SVMs** and **KNN** rely on manually crafted features, such as edges, textures, or shapes. While these models perform well on simpler tasks, they struggle with capturing the complex spatial hierarchies in images, which limits their scalability.
2. **Deep Learning:** On the other hand, **CNNs** can automatically learn complex hierarchical features directly from raw pixels. This means CNNs can recognize patterns at multiple levels, from simple edges to more complex shapes and objects. This gives them a distinct advantage over classical machine learning models.
3. **Advanced ML:** Techniques like **Neural Architecture Search (NAS)** and **self-supervised learning** are emerging as powerful methods. However, they require more computational resources and large datasets to be effective. In comparison, the CNN approach I used strikes a balance between accuracy, interpretability, and computational efficiency.

Strengths of this CNN approach include **high accuracy**, **stable convergence**, and **robust performance on unseen data**. However, there are some limitations, such as the absence of a **per-class F1 analysis**, which would have provided more detailed insights into the performance of each class. Additionally, the interpretability of CNNs is generally lower compared to attention-based or transformer models. Lastly, this model could potentially benefit from **transfer learning** or using **deeper architectures**.”

Slide 11: Conclusions and Lessons Learned

“To conclude, this project successfully demonstrated that a well-designed CNN can achieve a high accuracy of **91.03%** on the CIFAR-10 dataset. A few key lessons were learned during the process:

1. **Data augmentation** plays a significant role in improving model generalization, helping the model learn robust features that apply to new data.

2. **Batch normalization** helped stabilize the training process and accelerate convergence.
3. Techniques like **learning rate scheduling** and **dropout** were crucial for managing overfitting and ensuring that the model didn't memorize the training data.

For future work, I plan to experiment with more advanced architectures such as **ResNet** or **WideResNet**, which have shown great promise in image classification tasks. I also intend to explore **transfer learning** to leverage pretrained models from larger datasets, which could lead to even better performance.

Slide 12:

These are the references that I have used for my work please.

Thank you all for your attention. I'd be happy to answer any questions you may have."